

MINI PROJECT

ระบบจัดการรายการสิ่งที่ต้องทำ

Full-stack Web Application ด้วย Node.js, Express.js และ REST API

ผู้จัดทำ	เมธา ทองบุญนาก
รหัสนักศึกษา	660911119
หัวข้อทรัพยากร	To-do List / รายการสิ่งที่ต้องทำ
เทคโนโลยี	Node.js, Express.js, HTML, CSS, JavaScript, Fetch API

วัตถุประสงค์

- พัฒนาเว็บแอปพลิเคชันที่มีทั้งฝั่งเซิร์ฟเวอร์และฝั่งไคลเอนต์
- ออกแบบ REST API สำหรับเพิ่ม อ่าน แก้ไข ลบ และกรองข้อมูล
- เชื่อมต่อหน้าเว็บกับ API ด้วย fetch() โดยไม่โหลดหน้าใหม่
- ตรวจสอบข้อมูลและจัดการสถานะ 200, 201, 204, 400 และ 404 ให้เหมาะสม

ภาพรวมการทำงาน

เบราว์เซอร์ส่งคำขอ HTTP ไปยัง Express.js ผ่านเส้นทาง /api/tasks

เซิร์ฟเวอร์ตรวจสอบข้อมูลและจัดการรายการในหน่วยความจำ จากนั้นส่งผลลัพธ์ JSON กลับไปให้ JavaScript แสดงผลบนหน้าเว็บทันที

การออกแบบ REST API

ทรัพยากรหลักคือ tasks แต่ละรายการประกอบด้วย id, text, category และ done โดยรองรับการกรองด้วย Query String

Method	Endpoint	รายละเอียด	ผลสำเร็จ
GET	/api/tasks	อ่านรายการทั้งหมด และกรองด้วย done หรือ category	200
GET	/api/tasks/:id	อ่านรายการตาม ID; ตอบ 404 เมื่อไม่พบ	200
POST	/api/tasks	เพิ่มรายการใหม่; ตรวจสอบ text และ category	201
PATCH	/api/tasks/:id	แก้ไขชื่อหมวดหมู่ หรือสถานะ; ตอบ 404 เมื่อไม่พบ	200
DELETE	/api/tasks/:id	ลบรายการตาม ID	204

การตรวจสอบข้อมูล

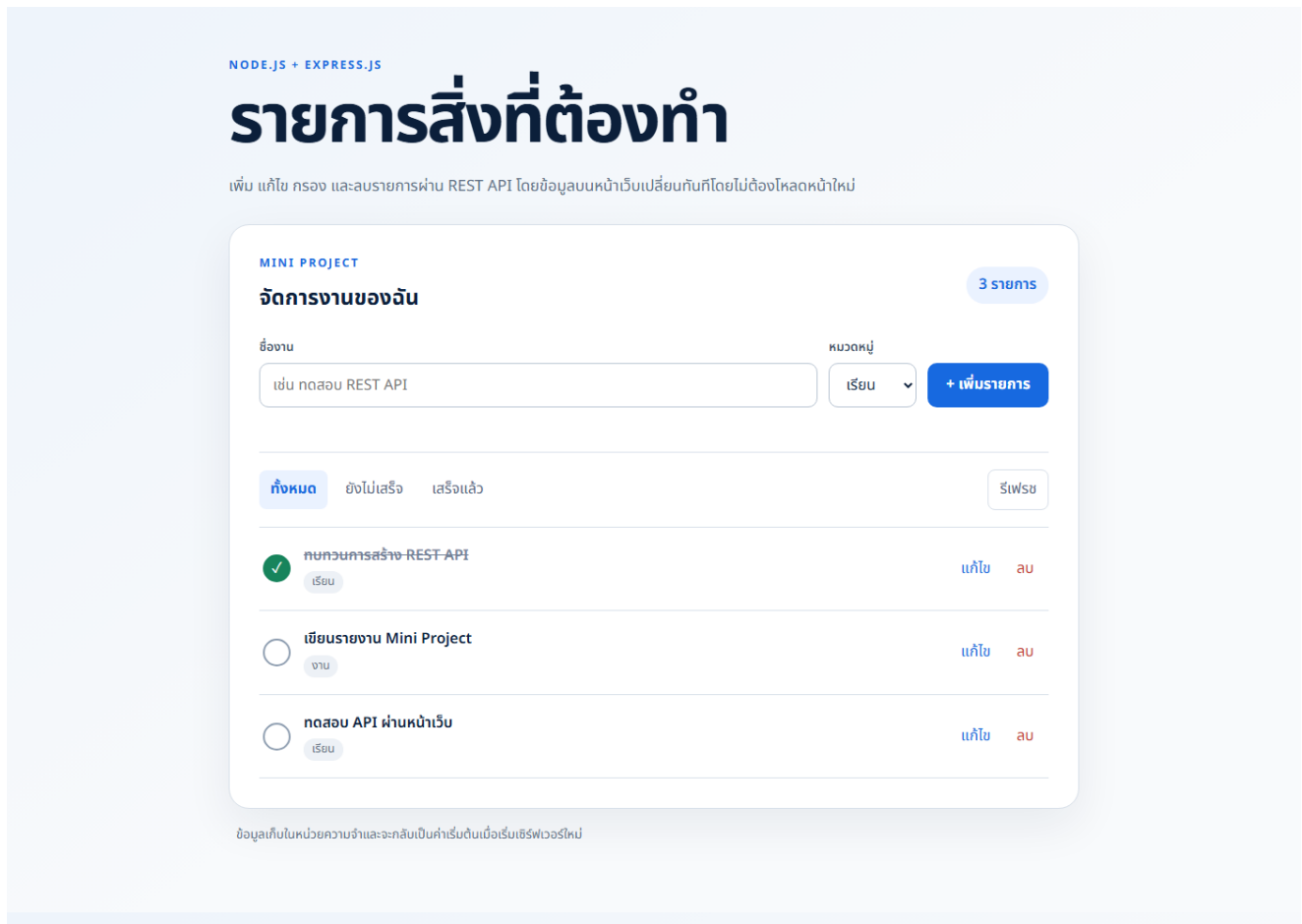
- POST ต้องมี text ที่ไม่ใช่ข้อความว่าง และยาวไม่เกิน 120 ตัวอักษร
- category ต้องเป็น เรียน, งาน หรือส่วนตัว
- done ต้องเป็นค่าบูลีน true หรือ false
- ข้อมูลผิดตอบ 400 และ ID ที่ไม่มีอยู่ตอบ 404 พร้อมข้อความ JSON

โครงสร้างโปรเจกต์

app.js ทำหน้าที่เป็นเซิร์ฟเวอร์และ API ส่วน public/index.html, public/style.css และ public/app.js เป็นหน้าเว็บฝั่งไคลเอนต์ เอกสารและภาพหลักฐานเก็บใน docs

หลักการทำงานของหน้าเว็บ

หน้าเว็บแสดงรายการจาก GET /api/tasks และมีแบบฟอร์มเพิ่มรายการ ปุ่มเปลี่ยนสถานะ ปุ่มแก้ไข และปุ่มลบ



ภาพที่ 1 หน้า Task Manager ซึ่งดึงข้อมูลจาก API และแสดงผลโดยไม่ต้องโหลดหน้าใหม่

ความสามารถของหน้าเว็บ

- แสดงสถานะกำลังโหลด ข้อผิดพลาด และกรณีไม่มีข้อมูล
- ตรวจสอบความก่อนส่ง POST
- แก้ไขข้อความและหมวดหมู่ผ่าน PATCH
- กรองรายการด้วย ?done=true หรือ ?done=false

หลักการเรียก API และผลการทดสอบ

ตัวอย่างนี้เรียก GET /api/tasks?done=false เพื่อให้เซิร์ฟเวอร์ส่งเฉพาะรายการที่ยังไม่เสร็จกลับมาเป็น JSON

```
pretty-print ☐
[{"id":2,"text":"เขียนรายงาน Mini Project","category":"งาน","done":false}, {"id":3,"text":"ทดสอบ API ผ่านหน้าเว็บ","category":"เรียน","done":false}]
```

ภาพที่ 2 ผลลัพธ์ JSON จาก API เมื่อใช้ Query String done=false

สรุปผลการทดสอบ

กรณีทดสอบ	สถานะที่ได้	ผล
GET /api/tasks?done=false	200	ผ่าน
GET ID ที่ไม่มีอยู่	404	ผ่าน
POST ข้อมูลว่าง	400	ผ่าน
POST ข้อมูลถูกต้อง	201	ผ่าน
PATCH แก้ไขข้อมูล	200	ผ่าน
DELETE รายการ	204	ผ่าน

สรุป

ระบบสามารถทำ CRUD ผ่าน REST API ได้ครบตามข้อกำหนด หน้าเว็บเรียก API ของโปรเจกต์เองด้วย fetch() และตอบสนองต่อสถานะการทำงานโดยไม่ต้องโหลดหน้าใหม่ ข้อมูลชุดนี้เหมาะสำหรับสาธิตแนวคิด Full-stack และการตรวจสอบ Request/Response